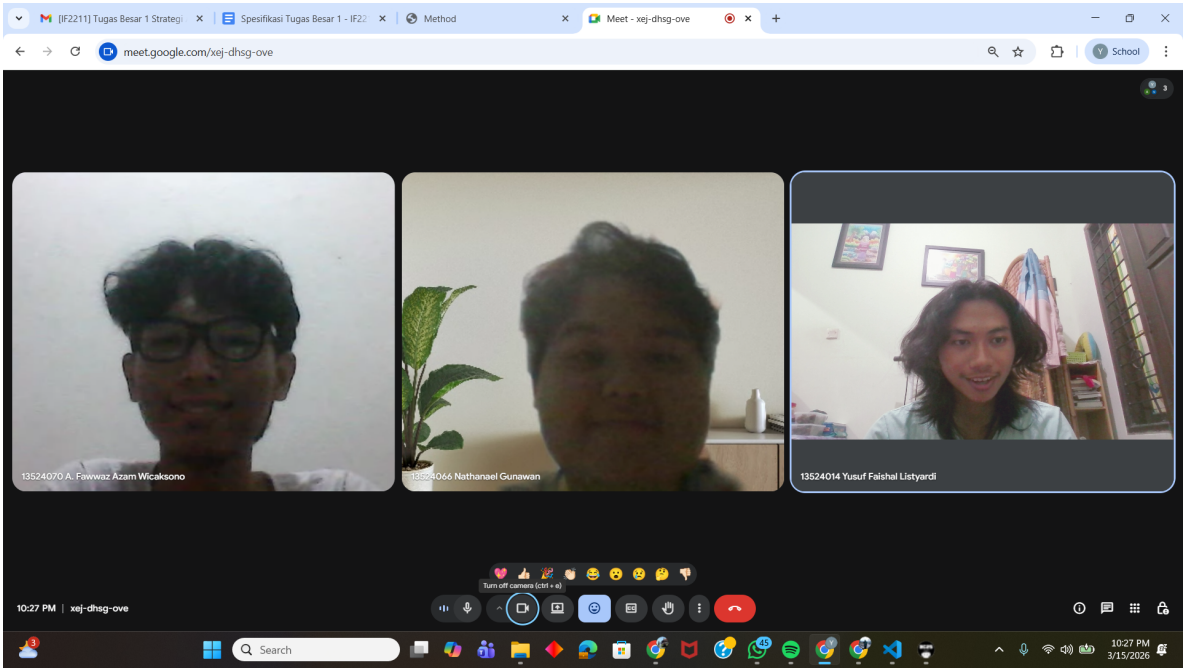
Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Pemanfaatan Algoritma Greedy Dalam Pembuatan Bot Permainan Battlecode 2025



AZZAM IS TIRED

1. Yusuf Faishal Listyardi (13524014)
2. Nathanael Gunawan (13524066)
3. A. Fawwaz Azam Wicaksono (13524070)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Masalah **4**

2 Dasar Teori **13**
2.1 Algoritma Greedy . . . 13
2.1.1 Definisi dan Prinsip Utama . . . 13
2.1.2 Properti Algoritma Greedy . . . 13
2.1.3 Prosedur Pemecahan Masalah . . . 13
2.1.4 Komponen Penyusun Algoritma . . . 13
2.1.5 Contoh Kasus . . . 14
2.1.6 Optimalitas Algoritma . . . 14
2.2 Bot . . . 14
2.2.1 Cara Kerja Bot . . . 14
2.2.2 Implementasi Bot . . . 15
2.2.3 Batasan Bytecode . . . 16
2.2.4 Cara Menjalankan Bot . . . 16
2.3 Game Engine . . . 16
2.3.1 Cara Kerja Game Engine . . . 17
2.3.2 Antarmuka Game Engine . . . 17

3 Aplikasi Strategi Greedy **18**
3.1 Abstraksi Permainan . . . 18
3.2 Eksplorasi Alternatif Solusi Greedy . . . 19
3.2.1 Penerapan 1: Ruinova . . . 19
3.2.2 Penerapan 2: Embege . . . 21
3.2.3 Penerapan 3: Strategi *Jack of All Trades* . . . 23
3.3 Pemilihan Strategi Greedy Utama . . . 24

4 Implementasi dan Pengujian **25**
4.1 Implementasi Algoritma Greedy pada Bot . . . 25
4.1.1 Ruinova . . . 25
4.1.2 Embege . . . 28
4.1.3 Jack of All Trades . . . 31
4.2 Struktur Data, Fungsi, dan Prosedur pada Strategi *Ruinova* . . . 41
4.2.1 Struktur Data yang Digunakan . . . 41
4.2.2 Fungsi Utama Program . . . 42
4.2.3 Prosedur Greedy Movement . . . 43
4.2.4 Hubungan Struktur Program dengan Strategi Greedy . . . 43
4.3 Pengujian . . . 44
4.3.1 *Ruinova* vs *Jack of All Trades* . . . 44
4.3.2 *Embege* vs *Jack of All Trades* . . . 45

4.3.3 *Ruinova* vs *Embege* 46

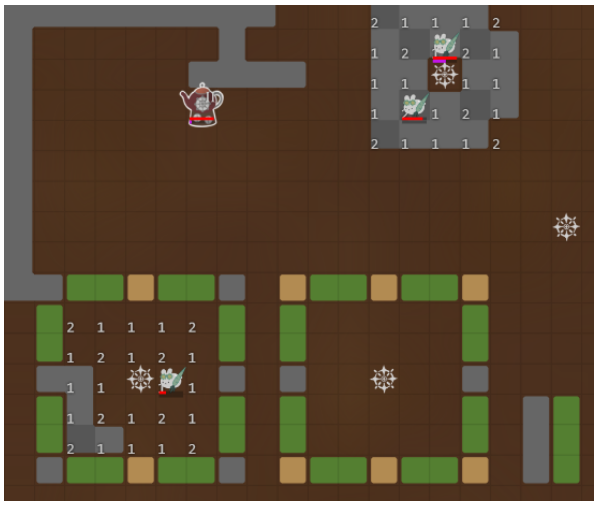
5 Penutup **47**
5.1 Kesimpulan 47
5.2 Saran dan Komentar 47
5.3 Pernyataan 48

6 Referensi **49**

7 Lampiran **50**

1 Deskripsi Masalah

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.



Gambar 1: Battlecode

Komponen komponen penting dari permainan Battlecode adalah sebagai berikut :

1. Objectives

   Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

2. Aturan Tiebreaker

   Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

   - Luas area yang sudah diwarnai di peta
   - Jumlah menara sekutu yang masih hidup
   - Total jumlah uang (chips)
   - Jumlah total cat (paint) yang dimiliki semua robot dan menara
   - Jumlah robot sekutu yang masih hidup
   - Jika masih seri, tim dipilih secara acak (uniform random)

3. Peta

   Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20×20 hingga 60×60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0, 0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek MapLocation yang menyimpan nilai x dan y. Agar tidak ada peta yang menguntungkan salah satu pemain, dijamin bahwa peta bersifat simetris (baik melalui rotasi maupun refleksi).

4. Jarak Pandang dan Pergerakan

   Robot selalu bisa melihat fitur peta dan robot lain dalam jarak hingga sqrt(20), kira kira 4.47 petak. Unit tidak bisa bergerak melewati menara atau robot lain. Peta mungkin memiliki tembok (walls), yang tidak bisa dilewati atau diwarnai. Tembok maksimal hanya 20% dari seluruh peta. Tembok tidak bisa dihapus atau dibangun oleh pemain. Robot juga tidak bisa melewati atau mewarnai reruntuhan (ruins).

5. Tower & Ruins

   Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh. Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (ruins), zona 5×5 yang sudah ditentukan di peta (lihat section towers untuk selengkapnya).

   Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan berjarak minimal 5 petak satu sama lain, dan zona 5×5 di sekitar pusat reruntuhan dijamin bebas tembok. Setiap tim memulai permainan dengan 2 menara: 1 menara cat dan 1 menara uang. Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

   Secara default, setiap tim memulai permainan dengan 1 menara penghasil uang dan 1 menara penghasil cat, keduanya sudah di-upgrade ke level 2.

6. Pola Sumber Daya Khusus

   Mewarnai pola khusus tertentu di peta dapat meningkatkan produksi sumber daya tim. Pola ini disebut Special Resource Pattern (SRP). Setiap pola aktif yang berhasil diwarnai akan membuat semua menara tambang sekutu mendapatkan +3 sumber daya tambahan per giliran.

   Robot bisa menandai pola ini menggunakan fungsi markResourcePattern(), lalu menyelesaikannya dengan completeResourcePattern() (biaya 200 chips). Setelah pola selesai dibuat, pola tersebut harus tetap utuh (tidak rusak warnanya) selama 50 ronde agar menjadi aktif. Pola ini sama untuk semua peta dan permainan.

7. Cat

   Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor. Setiap robot dan menara menyimpan cat secara individu, dengan kapasitas maksimal tergantung tipenya. Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi. Cat bisa diisi ulang dari mopper atau ditarik dari menara penghasil cat.

   Setiap tipe robot memiliki serangan berbeda yang bisa:

- Menggunakan cat untuk meletakkan warna di petak, atau
- Menghapus cat milik lawan dari petak.

Robot bisa memilih antara warna utama atau warna sekunder tim untuk mewarnai petak. Kedua warna ini dianggap sama untuk keperluan skor, tapi dibedakan saat mengecek penyelesaian pola cat.

8. Chips

   Chips digunakan untuk membangun unit & menara. Chips dihasilkan setiap ronde oleh menara penghasil uang, dan bisa digunakan oleh robot sekutu mana saja setelah dihasilkan. Setiap tim memulai permainan dengan 2500 chips.

9. Robot

   Semua robot dapat melakukan aksi seperti bergerak, merasakan (sensing), dan berkomunikasi satu sama lain. Dalam setiap pertempuran, robot milikmu akan berhadapan dengan satu tim musuh. Permainan berjalan berbasis giliran (turn-based) dan dibagi menjadi ronde. Pada setiap ronde, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. Kode yang dijalankan robot menghabiskan bytecode (ukuran sumber daya komputasi, dijelaskan lebih lanjut pada section bytecode limit). Setiap robot hanya memiliki jumlah bytecode tertentu per giliran; jika habis, gilirannya langsung berakhir dan akan dilanjutkan pada giliran berikutnya. Jika robot sudah selesai menjalankan aksinya pada giliran tersebut, ia harus memanggil **Clock.yield()** untuk menunggu giliran berikutnya dimulai.

   Semua robot memiliki HP (hitpoints / nyawa). Ketika HP mencapai nol, robot langsung dihapus dari permainan. Setiap robot memiliki ID unik yang diacak dan tidak kurang dari 10.000. Robot hanya dapat berinteraksi dengan lingkungan terdekat melalui sensing, pergerakan, dan kemampuan khusus. Setiap robot menjalankan salinan kode Anda secara independen. Robot tidak bisa berbagi variabel statis (masing-masing punya salinannya sendiri) karena dijalankan di JVM terpisah.

   Dua atau lebih robot tidak boleh berada di petak yang sama. Ketika cooldown pergerakan di bawah 10, robot dapat bergerak ke salah satu dari 8 petak tetangga. Semua robot memiliki cadangan cat (paint) dengan kapasitas maksimum tertentu. Semakin rendah cadangan cat, semakin tinggi cooldown pergerakan dan aksi yang dihadapi. Aturan penalti cat rendah:

   - Jika cadangan cat hanya X% penuh (X ¡ 50), maka cooldown meningkat (100 − 2X)%.
   - Jika cadangan cat habis sama sekali selama giliran, robot tidak bisa bergerak atau melakukan aksi apa pun (kecuali self-destruct) sampai mendapat cat lagi, dan akan kehilangan 20 HP per ronde.

   Robot spawn dengan cadangan cat 100% penuh dan dapat diisi ulang di menara atau oleh mopper.

   Kehilangan cat saat mengakhiri giliran:

   - Di wilayah netral : kehilangan 1 cat

- Di wilayah musuh : kehilangan 2 cat
- Di wilayah sekutu : tidak kehilangan cat

Penalti cat tambahan setiap ronde adalah 1 × jumlah robot sekutu yang bersebelahan (dalam 8 petak sekitar). Pinalti ini berlipat ganda (×2) saat berada di wilayah musuh. Robot dapat di-spawn dalam radius aksi $\sqrt{4}$ dari menara sekutu, dengan biaya cat dan chips sesuai jenisnya.

10. Jenis Robot

    - Soldier
      - HP: 250
      - Kapasitas cat maks: 200
      - Biaya produksi: 200 cat + 250 chips
      - Serangan: Menyerang satu petak (radius aksi 3)
      - Jika petak kosong atau sudah ada cat sekutu → mewarnai petak tersebut
      - Menimbulkan 50 damage ke menara musuh (tidak ke robot)
      - Biaya serangan: 5 cat
      - Cooldown aksi: 10
    - Splasher
      - HP: 150
      - Kapasitas cat maks: 300
      - Biaya produksi: 300 cat + 400 chips
      - Serangan: Menyerang area melingkar (radius 2 dari titik pusat)
      - Titik pusat serangan maksimal 2 petak dari posisi splasher
      - Dalam radius $\sqrt{2}$ (1.41) dari pusat → bisa menimpa cat musuh
      - Mewarnai semua petak kosong / sekutu di area
      - Menimbulkan 100 damage ke menara musuh (tidak ke robot)
      - Biaya serangan: 50 cat
      - Cooldown aksi: 50
    - Mopper
      - HP: 50
      - Kapasitas cat maks: 100
      - Biaya produksi: 100 cat + 300 chips
      - Kemampuan utama:
        * Mop petak (radius $\sqrt{2}$)
          · Menghapus cat musuh di petak tersebut
          · Jika ada robot musuh di petak → musuh kehilangan 10 cat, mopper mendapat 5 cat
          · Cooldown: 30
        * Transfer cat ke robot atau menara sekutu (radius $\sqrt{2}$)
          · Mentransfer jumlah cat tertentu dari cadangan sendiri
          · Cooldown: 10

    * Ayunan mop (swing) ke salah satu dari 4 arah utama (atas, bawah, kiri, kanan)
      · Menghapus 5 cat dari masing-masing robot musuh yang berdekatan dalam garis lurus
      · Serangan diterapkan dua kali di arah yang sama: → 1 langkah ke arah tersebut → 2 langkah ke arah yang sama
      · Maksimal 6 target potensial (3 di langkah 1 + 3 di langkah 2)
      · Cooldown: 20

    Catatan tambahan: Mopper mendapat penalti cat ganda saat bergerak atau mengakhiri giliran di wilayah musuh.

11. Towers

- Prosedur Pembangunan Menara

  Semua menara dibangun dengan prosedur yang sama. Menara hanya bisa dibangun di atas reruntuhan (ruins), setelah pola cat 5×5 tertentu ditempatkan di sekitar reruntuhan tersebut.

  – Panggil markTowerPattern() untuk secara otomatis menempatkan penanda di area 5×5 sekitar reruntuhan. Penanda ini menunjukkan posisi mana yang harus diwarnai dengan warna utama (primary color).
  – Setelah pola sudah diwarnai, robot bisa memanggil completeTowerPattern() untuk memunculkan menara yang ditentukan di lokasi reruntuhan.

- Posisi dan Ukuran Menara Menara yang dihasilkan akan menempati blok 1×1 di tengah bentuk spawn. Bentuk berbeda menghasilkan jenis menara berbeda dengan rentang serangan, laju serang, laju penambangan, dan kesehatan yang berbeda (lihat tabel di bawah).

  Level dan Upgrade

  – Menara selalu dibangun di level 1
  – Bisa di-upgrade ke level lebih tinggi menggunakan upgradeTower() dalam radius sqrt(2) (1.41 petak).

  Batasan

  – Tidak ada tim yang boleh memiliki lebih dari 25 menara sekaligus.
  – Setiap menara bisa menyimpan 1000 cat.

  Batasan dan Menara Pertahanan Setiap menara pertahanan yang hidup meningkatkan kekuatan serangan satu blok (single-block attack) dari semua menara sekutu sebesar:

  – +5 (level 1)
  – +7 (level 2)
  – +9 (level 3)

  Bonus ini tidak memengaruhi serangan area (AoE) menara sekutu.

Name	Build/ Upgrade cost	Attack Range	Single-block Attack Strength	AoE Attack Strength	Base Mining Rate	Health
Money Lv1	1000 chips	3	20	10	20 chips/turn	1000
Lv2	2500 chips	3	20	10	30 chips/turn	1500
Lv3	5000 chips	3	20	10	40 chips/turn	2000
Paint, Lv1	1000 chips	3	20	10	5 paint/turn	1000
Lv2	2500 chips	3	20	10	10 paint/turn	1500
Lv3	5000 chips	3	20	10	15 paint/turn	2000
Defense, Lv1	1000 chips	4	40	20	20 chips/attack*	2000
Lv2	2500 chips	4	50	25	30 chips/attack*	2500
Lv3	5000 chips	4	60	35	40 chips/attack*	3000

Gambar 2: Ability Lengkap Tower

12. Serangan

    Menara dapat menyerang unit musuh. Menara memiliki dua jenis serangan:

    - Serangan satu petak (single-block attack) yang menyerang satu unit musuh di satu petak.
    - Serangan area efek (AoE / area of effect) jarak jauh yang memberikan kerusakan kepada semua unit musuh di dalam jangkauannya.

    Setiap giliran, satu menara dapat melakukan satu serangan satu petak dan satu serangan area efek.

13. Penambangan

    Setiap giliran, menara penghasil uang dan menara penghasil cat akan secara pasif menambang sumber daya.

    - Cat disimpan di dalam menara dan dapat diambil oleh robot sekutu.
    - Uang (chips) tersedia untuk semua unit sekutu.

    Setiap menara memulai dengan 500 cat untuk digunakan spawn berbagai jenis unit. Menara dapat memunculkan robot dalam jarak 2 petak dari posisinya. Jika cat di menara habis, menara tidak akan bisa spawn unit lagi sampai diisi ulang atau menghasilkan cat sendiri. Cat dapat diisi ulang menggunakan cat yang dikumpulkan oleh mopper.

14. Komunikasi

    Robot hanya bisa melihat lingkungan di sekitarnya secara langsung dan dikendalikan secara independen oleh salinan kode yang sama, sehingga koordinasi antar-robot sangat sulit. Anda tidak dapat berbagi variabel apa pun di antara mereka; bahkan variabel static pun tidak akan dibagikan karena setiap robot memiliki salinannya sendiri.

    Komunikasi dilakukan dengan dua cara:

    - Pesan (Messages) Pesan dapat berisi informasi apa saja, tetapi dibatasi hanya 4 byte.

- Penanda (Markers) Penanda menyatakan warna apa yang seharusnya digunakan untuk mewarnai ulang suatu petak, dan penanda ini terlihat oleh semua robot sekutu. Penanda tidak memberikan efek lain apa pun dalam permainan.

Pesan Robot atau menara yang berada dalam jangkauan satu sama lain dapat saling mengirim pesan. Sebuah robot dianggap "dalam jangkauan" (in range) sebuah menara jika memenuhi dua syarat sekaligus:

- Jaraknya ke menara tersebut $\leq$ sqrt(20) (4.47 petak), dan
- Robot tersebut terhubung dengan menara melalui jalur cat sekutu (artinya robot bisa bergerak sampai ke menara hanya dengan berjalan di petak-petak yang sudah memiliki cat milik timnya sendiri).

Setiap robot dan menara menyimpan semua pesan yang diterima dalam 5 ronde terakhir di dalam buffer-nya. Setelah 5 ronde berlalu, pesan tersebut akan dihapus secara otomatis.

Batas Pengiriman Pesan per Ronde adalah sebagai berikut

- Robot: maksimal 1 pesan per ronde
- Menara: maksimal 20 pesan per ronde

Setiap pesan terdiri dari:

- Satu bilangan bulat 32-bit (integer)
- Nomor ronde saat pesan dikirim
- ID robot yang mengirim pesa

Menara memiliki kemampuan tambahan untuk melakukan broadcast (siaran) pesan ke menara sekutu lain yang berada dalam jarak sqrt(80) (8.94 petak).

- Menara tidak perlu terhubung melalui jalur cat untuk melakukan broadcast ini.
- Setiap kali broadcast dilakukan, itu dihitung sebagai satu pesan terhadap batas 20 pesan per ronde menara.
- Kemampuan ini memungkinkan menara memperluas jangkauan komunikasi jarak jauh antar-menara.

Penanda Penanda adalah informasi yang bisa ditempatkan atau dihapus di peta oleh robot. Penanda hanya memiliki dua jenis:

- Menandai dengan warna sekunder
- Menandai dengan warna utama

Penanda hanya terlihat oleh robot sekutu dan akan tetap ada di petak tersebut sampai dihapus atau diganti dengan penanda lain oleh sekutu. Selain itu, penanda hanya dapat ditempatkan pada ubin yang dapat dicat

Penanda dapat diletakkan dengan cara

- Secara manual: robot dapat menempatkan penanda di petak yang berjarak $\leq$ sqrt(2) (1.41 petak) dari posisi robot saat ini. Akan dijelaskan lebih lanjut pada section Action
- Secara otomatis melalui fungsi pola:
  - markTowerPattern() $\rightarrow$ menandai pola 5×5 di sekitar reruntuhan dengan penanda yang sesuai untuk membangun menara
  - markResourcePattern() $\rightarrow$ menandai pola 5×5 untuk Special Resource Pattern (SRP)

15. Action and Cooldown

    Robot melakukan berbagai aksi untuk berinteraksi dengan dunia permainan. Beberapa aksi tidak bisa dilakukan berkali-kali dalam satu giliran atau dalam waktu singkat karena terikat cooldown. Berikut daftar aksi utama:

    - Menyerang/mewarnai

      Menyerang dan mewarnai dilakukan dengan fungsi yang sama. Aksi ini hanya bisa dilakukan jika action cooldown robot ¡ 10. Action cooldown robot dapat dicek dengan fungsi isActionReady(). Dilakukan dengan memanggil metode attack() yang akan menaargetkan satu petak yang dapat dilihat oleh robot tersebut. Efeknya tergantung tipe robot (seperti yang dijelaskan di bagian Robot): memberikan damage atau mewarnai petak. Setelah dilakukan, action cooldown robot bertambah sesuai nilai yang ditentukan oleh tipe robot.
    - Bergerak(moving)

      Hanya bisa dilakukan jika movement cooldown ¡ 10. Movement cooldown dapat dicek dengan isMovementReady(). Petak tujuan harus kosong (tidak ditempati unit lain) dan bisa dilewati (passable). Dilakukan dengan memanggil move() dengan arah yang diinginkan. Setelah bergerak, movement cooldown bertambah 10. Untuk memeriksa apakah gerakan valid, gunakan canMove().
    - Menandai (marking)

      Semua robot bisa menandai petak tempat robot sedang berdiri, atau Menghapus penanda sekutu dari petak tempat robot berdiri. Biaya dari aksi ini adalah 1 cat. Aksi ini tidak menambah action cooldown sama sekali. Aksi dapat dilakukan dengan mark() atau removeMark(). Penanda tidak memengaruhi kepemilikan wilayah (tidak membuat petak menjadi wilayah sekutu). Penanda hanya bisa dirasakan oleh robot sekutu melalui senseNearbyMapInfos(). Penanda secara umum tersebut hanya boleh ditempatkan pada petak yang dapat diwarnai (tidak ada tembok atau reruntuhan). Selain itu, terdapat Fungsi khusus penanda, yakni markTowerPattern() dan markResourcePattern()menggunakan 25 cat dan menandai area 5×5 dengan pola yang sesuai.
    - Mentransfer Cat (Transferring)

      Hanya dapat dilakukan oleh robot mopper, dan hanya dapat dilakukan jika action cooldown ¡ 10 (dapat dicek dengan isActionReady()). Aksi ini dapat dilakukan dengan memanggil prosedur transferPaint(). Jarak maksimal: $\sqrt{2}$ (1.41 petak). Mentransfer jumlah cat tertentu ke target (robot atau menara

sekutu). Jika jumlah yang diberikan negatif, maka mopper justru mengambil cat dari target. Setelah dilakukan, action cooldown bertambah sesuai nilai yang disebutkan di deskripsi mopper.

- Menarik cat dari menara (withdrawing)

  Semua robot bisa menarik cat dari menara sekutu untuk mengisi ulang cadangan cat mereka. Memerlukan action cooldown ¡ 10. Setelah dilakukan, action cooldown bertambah 10. Jarak maksimal: $\sqrt{2}$ (1.41 petak). Dilakukan dengan transferPaint() menggunakan nilai negatif untuk menunjukkan jumlah yang ingin ditarik.

- Spawn Robot (spawning)

  Hanya bisa dilakukan jika action cooldown menara ¡ 10 (dicek dengan isActionReady()). Dilakukan dengan buildRobot(). Mengurangi biaya robot (cat + chips) dari stok menara. Memunculkan satu robot tipe tertentu di petak yang ditentukan. Setelah dilakukan, action cooldown menara bertambah 10.

- Berkomunikasi (communitating)

  Robot menggunakan sendMessage() untuk mengirim pesan ke menara sekutu dalam jangkauan. Menara menggunakan sendMessage() untuk mengirim pesan ke robot sekutu dalam jangkauan. Robot menggunakan readMessages() untuk membaca pesan baru yang diterima dalam 5 ronde terakhir dari buffer. Kedua metode ini tidak menambah action cooldown. Syarat komunikasi: kedua unit harus berada di petak yang terhubung melalui jalur cat sekutu.

- Menghancurkan Diri Sendiri (Disintegrating)

  Robot apa pun bisa memanggil disintegrate(). Menghancurkan robot tersebut secara instan. Tidak ada cooldown khusus untuk aksi ini.

2 Dasar Teori

2.1 Algoritma Greedy

2.1.1 Definisi dan Prinsip Utama

Algoritma greedy merupakan metode yang digunakan untuk memecahkan persoalan optimasi, baik itu maksimasi maupun minimasi. Prinsip algoritma greedy adalah "take what you can get now!".

Solusi dibentuk secara langkah demi langkah (step by step), di mana pada setiap langkah dibuat keputusan terbaik tanpa memperhatikan konsekuensinya di masa depan. Proses ini dilakukan dengan harapan optimum lokal pada setiap langkah akan membuahkan optimum global.

2.1.2 Properti Algoritma Greedy

Sebuah persoalan dapat diselesaikan secara optimal menggunakan algoritma greedy jika memenuhi dua properti berikut:

- Greedy Choice Property: Solusi optimal global dapat dibangun dengan memilih keputusan optimal lokal pada setiap langkahnya.
- Optimal Substructure: Solusi optimal dari persoalan besar merupakan gabungan dari solusi-solusi optimal dari sub-persoalannya.

2.1.3 Prosedur Pemecahan Masalah

Untuk merumuskan dan membuktikan bahwa algoritma greedy dapat menyelesaikan sebuah persoalan, terdapat prosedur standar yang umumnya dilakukan:

- Rumusan persoalan optimasi dengan pilihan greedy: Merumuskan persoalan sebagai suatu proses membuat pilihan greedy secara lokal dan menyisakan sub-persoalan untuk diselesaikan.
- Menunjukkan adanya optimal substructure: Membuktikan bahwa dengan menggabungkan solusi optimal dari sub-persoalan melalui pendekatan greedy, akan diperoleh solusi optimal untuk persoalan semula.
- Membuktikan solusi greedy-optimal: Membuktikan secara matematis bahwa selalu ada solusi optimal untuk persoalan semula dengan mengambil pilihan greedy pada setiap langkahnya.

2.1.4 Komponen Penyusun Algoritma

Dalam perancangannya, algoritma greedy umumnya melibatkan enam komponen utama:

- Himpunan Kandidat (C): Berisi elemen-elemen yang akan dipilih pada setiap langkah (misalnya koin, job, atau sisi graf).
- Himpunan Solusi (S): Berisi kandidat yang telah terpilih sebagai bagian dari solusi.

- Fungsi Seleksi: Memilih kandidat terbaik berdasarkan strategi heuristik tertentu (misalnya memilih koin terbesar atau durasi tercepat).
- Fungsi Kelayakan (Feasible): Memeriksa apakah kandidat yang dipilih tidak melanggar batasan (constraint) yang ada jika dimasukkan ke dalam solusi.
- Fungsi Solusi: Menentukan apakah himpunan solusi yang terbentuk sudah lengkap.
- Fungsi Objektif: Mengukur nilai optimasi yang ingin dicapai (maksimal atau minimal).

2.1.5 Contoh Kasus

Untuk memperkuat teori, Anda bisa menyebutkan beberapa algoritma populer yang menggunakan pendekatan greedy, antara lain:

- Penukaran Uang (Coin Exchange): Memilih koin terbesar untuk meminimalkan jumlah koin.
- Pohon Merentang Minimum (Minimum Spanning Tree): Menggunakan Algoritma Prim atau Kruskal.
- Lintasan Terpendek (Shortest Path): Menggunakan Algoritma Dijkstra.
- Kode Huffman: Digunakan untuk pemampatan data yang efisien.

2.1.6 Optimalitas Algoritma

Algoritma greedy tidak selalu menjamin pencapaian solusi optimal. Hal ini dapat terjadi karena algoritma greedy tidak mengevaluasi seluruh kemungkinan solusi yang ada secara menyeluruh (seperti pada metode exhaustive search), dan ada kemungkinan fungsi seleksi (greedy choice) yang dirancang tidak mengarah pada hasil optimal. Apabila algoritma ini tidak berhasil memberikan solusi optimal yang mutlak, maka solusi yang dihasilkan dianggap sebagai solusi hampiran (pseudo-optimum atau sub-optimum)

2.2 Bot

Bot adalah program perangkat lunak yang bekerja secara otomatis untuk melakukan suatu tugas berulang. Bot mengikuti instruksi pada kode yang telah diprogram dan menjalankannya terus-menerus sampai ada kondisi berhenti yang terpenuhi. Dalam konteks permainan Battlecode 2025, bot adalah program Java yang mengendalikan setiap unit dan menara secara independen untuk mencapai tujuan permainan.

2.2.1 Cara Kerja Bot

Setiap unit (Soldier, Splasher, Mopper) dan setiap menara dalam Battlecode 2025 menjalankan salinan kode yang sama secara independen. Artinya, tidak ada *shared state* langsung antar unit — bahkan variabel `static` pun tidak dibagikan karena setiap robot dijalankan pada JVM yang terpisah.

Setiap giliran (*turn*), sebuah bot menjalankan siklus berikut:

1. **Sense (Persepsi):** Bot membaca kondisi lingkungan sekitarnya menggunakan fungsi-fungsi sensing yang disediakan API, seperti mendeteksi robot musuh, menara, petak yang belum diwarnai, serta informasi sumber daya yang tersedia.

2. **Think (Pemrosesan):** Bot memproses informasi yang diperoleh untuk mengambil keputusan aksi terbaik pada giliran tersebut. Pada tahap inilah algoritma greedy diterapkan — bot memilih aksi yang paling menguntungkan secara lokal berdasarkan kondisi saat itu.

3. **Act (Eksekusi):** Bot mengirimkan perintah ke *game engine*, seperti bergerak (`move()`), menyerang atau mewarnai petak (`attack()`), membangun unit baru (`buildRobot()`), atau menarik cat dari menara (`transferPaint()`).

4. **Yield:** Setelah aksi selesai atau batas *bytecode* mendekati habis, bot mengakhiri gilirannya dengan memanggil `Clock.yield()` agar giliran dilanjutkan pada ronde berikutnya.

2.2.2 Implementasi Bot

Seluruh logika bot diimplementasikan dalam satu file utama bernama `RobotPlayer.java` yang berada dalam *package* bot masing-masing. File ini wajib memiliki method statis `run(RobotController rc)` yang dipanggil oleh *game engine* pada setiap giliran robot. Struktur direktori bot mengikuti template yang disediakan oleh asisten sebagai berikut:

```
src/
 main-bot/
    RobotPlayer.java
 alternative-bots-1/
    RobotPlayer.java
 alternative-bots-2/
     RobotPlayer.java
```

Terdapat beberapa kelas dan antarmuka utama dari API Battlecode yang digunakan dalam implementasi bot, antara lain:

- `RobotController`: Kelas inti yang menjadi antarmuka antara kode bot dengan *game engine*. Semua aksi dan sensing dilakukan melalui objek `rc` bertipe `RobotController`.

- `RobotInfo`: Menyimpan informasi tentang robot yang terdeteksi di sekitar bot, seperti tipe, tim, posisi, dan jumlah HP.

- `MapLocation`: Merepresentasikan koordinat (x, y) suatu petak di peta permainan.

- `RobotType`: Enum yang merepresentasikan tipe robot, yaitu `SOLDIER`, `SPLASHER`, dan `MOPPER`.

- `Clock`: Digunakan untuk mengecek sisa *bytecode* pada giliran saat ini melalui `Clock.getBytecodesLeft()` dan mengakhiri giliran dengan `Clock.yield()`.

2.2.3 Batasan Bytecode

Setiap robot memiliki batas komputasi per giliran yang disebut *bytecode limit*. Satu *bytecode* kira-kira setara dengan satu operasi dasar dalam Java. Batas *bytecode* yang ditetapkan adalah 17.500 untuk semua robot dan 20.000 untuk semua menara. Jika sebuah robot melebihi batas ini, eksekusinya akan dihentikan sementara dan dilanjutkan pada giliran berikutnya yang dapat menyebabkan perilaku bot tidak terduga.

Adanya batasan ini menjadi salah satu alasan utama digunakannya algoritma *greedy* dalam implementasi bot. Algoritma *greedy* yang hanya mengevaluasi kondisi saat ini tanpa eksplorasi menyeluruh ke depan memiliki kompleksitas komputasi yang jauh lebih rendah dibandingkan pendekatan lain seperti *exhaustive search* atau pemrograman dinamis, sehingga lebih aman dari risiko melampaui batas *bytecode*.

2.2.4 Cara Menjalankan Bot

Untuk menjalankan bot yang telah diimplementasikan, dibutuhkan beberapa *dependencies*, yaitu Java Development Kit (JDK) versi 8 atau lebih tinggi, Gradle, dan Git.

Setelah semua *dependencies* terpenuhi, lakukan langkah-langkah berikut:

1. Clone *repository* yang disediakan asisten dengan perintah:

   ```
   git clone https://github.com/Fariz36/STIMA-battle
   cd STIMA-battle
   ```

2. Lakukan *build* pada proyek dengan perintah:

   ```
   ./gradlew build
   ```

3. Masuk ke direktori `client` dan jalankan aplikasinya:

   ```
   cd client
   ```

4. Setelah aplikasi terbuka, pilih direktori `STIMA-battle` (bukan `STIMA-battle/src`) sebagai *root directory*.

5. Pilih bot yang ingin dipertandingkan, atur konfigurasi pertandingan, lalu mulai permainan.

2.3 Game Engine

Game engine merupakan fondasi dari permainan Battlecode 2025 yang menjadi tempat seluruh bot bertarung. *Game engine* bertanggung jawab atas simulasi permainan: mengelola giliran setiap robot, memproses aksi yang dikirimkan bot, memperbarui kondisi peta, serta menentukan pemenang berdasarkan aturan yang berlaku.

Game engine yang digunakan pada Tugas Besar ini telah dimodifikasi oleh asisten dan dapat diakses melalui *repository* berikut: `https://github.com/Fariz36/STIMA-battle`.

2.3.1 Cara Kerja Game Engine

Permainan Battlecode 2025 berjalan secara *turn-based* dan dibagi menjadi *round*. Pada setiap *round*, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. *Game engine* bekerja dengan alur sebagai berikut:

1. *Game engine* menginisialisasi arena berupa peta grid persegi panjang berukuran antara 20×20 hingga 60×60 petak.
2. Setiap tim memulai permainan dengan 2 menara awal (1 menara cat dan 1 menara uang) serta 2.500 *chips* sebagai modal awal.
3. Pada setiap *round*, *game engine* memanggil method `run(rc)` pada setiap robot secara bergiliran. Bot kemudian mengeksekusi logikanya dan mengirimkan perintah aksi kepada *engine*.
4. *Game engine* memvalidasi setiap perintah yang diterima (misalnya memeriksa apakah robot memiliki cukup *action cooldown*, cukup cat, atau apakah petak tujuan bisa dilewati), lalu menerapkan aksi yang valid ke dalam kondisi permainan.
5. Permainan berakhir jika salah satu tim berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, atau salah satu tim kehilangan seluruh robot dan menaranya, atau ronde ke-2000 telah tercapai.

2.3.2 Antarmuka Game Engine

Game engine menyediakan antarmuka grafis (*GUI*) berupa Battlecode Client yang memungkinkan pengguna untuk memilih bot yang akan dipertandingkan, mengatur konfigurasi peta dan aturan permainan, serta memvisualisasikan jalannya pertandingan secara *real-time*.

3 Aplikasi Strategi Greedy

3.1 Abstraksi Permainan

Permainan ini dapat diabstraksikan sebagai sebuah graf grid dua dimensi $G = (V, E)$, di mana himpunan simpul V merepresentasikan petak-petak (koordinat) pada peta, dan himpunan sisi E merepresentasikan keterhubungan antar petak yang bertetangga. Setiap simpul $v \in V$ memiliki beberapa atribut (state), antara lain:

- Status kepemilikan cat (netral, kawan, atau lawan).
- Entitas yang menempatinya (kosong, rintangan, Tower, atau Unit Bot).

Permainan dimodelkan secara *turn-based*. Pada setiap gilirannya, entitas yang dimiliki (Tower dan 3 jenis Bot) mengambil keputusan secara otonom berdasarkan keterbatasan sumber daya (paint/resource) dan status *cooldown* aksi. Tujuan akhir pencapaian mewarnai lebih dari 70% peta dioptimasi melalui serangkaian keputusan lokal terbaik pada setiap giliran.

Berdasarkan abstraksi permainan yang telah dijelaskan, dilakukan *mapping* persoalan *Battlecode 2025* ke dalam elemen-elemen algoritma *greedy*. Setiap giliran (*turn*), bot dihadapkan pada sejumlah aksi yang dapat dipilih sesuai kondisi lokal di sekitarnya. Karena keputusan harus diambil secara cepat dengan keterbatasan *cooldown*, sumber daya, dan radius aksi, maka strategi *greedy* digunakan untuk memilih aksi lokal terbaik pada setiap giliran. Adapun pemetaan elemen-elemen *greedy* pada permainan ini adalah sebagai berikut.

a. **Himpunan Kandidat (C)**
   Himpunan kandidat dalam permainan ini adalah seluruh aksi yang dapat dipertimbangkan oleh entitas pada satu giliran, meliputi:
   - Bergerak ke petak tetangga yang dapat dilalui.
   - Mewarnai petak dalam radius serangan.
   - Menghapus cat lawan dari petak tertentu.
   - Mentransfer atau mengambil *paint*.
   - Menyerang unit atau wilayah lawan.
   - Membangun unit baru atau menyelesaikan pola menara.

b. **Himpunan Solusi (S)**
   Himpunan solusi merupakan urutan aksi dari himpunan kandidat yang dipilih secara bertahap untuk mencapai dominasi wilayah, antara lain:
   - Jalur pergerakan menuju area netral atau area lawan.
   - Rangkaian aksi pewarnaan untuk memperluas wilayah tim.
   - Kombinasi aksi pembangunan dan pertahanan area sekitar *tower*.
   - Pola distribusi unit untuk menjaga kontrol wilayah.

c. **Fungsi Seleksi**
   Fungsi seleksi digunakan untuk memilih kandidat aksi yang memberikan keuntungan langsung terbesar, dengan prioritas:

- Petak netral yang dapat segera dikuasai.
- Petak lawan yang bernilai strategis untuk direbut.
- Posisi yang membuka akses ke area baru.
- Target yang dapat dicapai dengan penggunaan *paint* minimum.

d. **Fungsi Kelayakan**
Fungsi kelayakan menentukan apakah kandidat aksi dapat dijalankan sesuai aturan permainan, yaitu:

- Aksi berada dalam radius gerak atau radius serangan.
- Unit memiliki *paint* yang cukup.
- Status *cooldown* memungkinkan aksi dilakukan.
- Petak tujuan tidak terhalang rintangan atau unit lain.

e. **Fungsi Objektif**
Fungsi objektif dalam permainan ini adalah memaksimalkan kontrol wilayah untuk memenangkan permainan, dengan tujuan:

- Memperbesar jumlah petak berwarna milik tim.
- Mengurangi dominasi wilayah lawan.
- Menjaga efisiensi penggunaan sumber daya.
- Mempertahankan posisi strategis hingga akhir permainan.

3.2 Eksplorasi Alternatif Solusi Greedy

Karena kompleksitas aksi yang dapat dilakukan, strategi greedy tidak diterapkan secara monolitik, melainkan dipecah menjadi beberapa fungsi optimasi berdasarkan jenis entitas dan fase aksinya.

3.2.1 Penerapan 1: Ruinova

Algoritma pada program ini menggunakan pendekatan heuristic *greedy by local priority* dengan koordinasi sederhana antara *Tower*, *Soldier*, dan *Splasher*. Heuristik utama diberi nama *Ruinova*, yang berasal dari kata *ruin* dan *nova*, karena strategi utamanya menjadikan *ruin* sebagai titik awal ekspansi wilayah dan pembangunan infrastruktur baru. Setiap unit mengambil keputusan secara mandiri berdasarkan kondisi lokal yang terdeteksi pada giliran tersebut.

1. Peran Tower

   Tower berfungsi sebagai pusat produksi unit dan pertahanan awal. Pada setiap *turn*, *Tower* terlebih dahulu memeriksa apakah terdapat musuh dalam radius serang. Jika ada target yang dapat diserang, maka aksi serangan dilakukan lebih dahulu.

   Setelah itu, *Tower* menentukan jenis unit yang akan diproduksi menggunakan heuristic berbasis fase permainan. Pada fase awal (sebelum ronde 400), komposisi produksi didominasi *Soldier* dengan rasio tiga banding satu terhadap *Splasher*. Pada fase menengah (ronde 400–800), proporsi *Splasher* ditingkatkan. Setelah

jumlah *tower* mencukupi, produksi semakin difokuskan pada *Splasher*. Selain produksi unit, *Tower* juga melakukan *upgrade* setelah ronde ke-300 apabila sumber daya memungkinkan.

2. Peran Soldier

   Soldier berfungsi sebagai unit utama ekspansi wilayah dan pembangunan *tower*. Pada awal giliran, unit terlebih dahulu memeriksa ketersediaan *paint*. Jika jumlah *paint* rendah, *Soldier* mengambil *paint* dari *Tower* terdekat agar tetap dapat melanjutkan tugas.

   Selanjutnya, *Soldier* memindai area sekitar untuk mencari *ruin*. Jika ditemukan *ruin* yang belum ditempati *tower*, lokasi tersebut langsung dipilih sebagai target utama. Setelah mendekati target, unit mulai menandai pola *tower*, mengecat petak yang belum sesuai, lalu menyelesaikan pembangunan struktur. Jika ditemukan *ruin* yang belum ditempati *tower*, lokasi valid pertama yang terdeteksi langsung dipilih sebagai target utama.

3. Peran Splasher

   Splasher berfungsi sebagai unit pembersih wilayah lawan. Pada setiap giliran, unit mencari petak musuh di sekitar area pengamatan dan mengevaluasi target berdasarkan kepadatan cat musuh di sekitarnya. Setiap kandidat target diberi skor berdasarkan jumlah cat musuh di radius kecil sekitar target, dikurangi jarak terhadap posisi unit.

   Target dengan kepadatan tertinggi diprioritaskan karena memberikan keuntungan wilayah terbesar dalam satu serangan. Jika target dapat dijangkau, *Splasher* langsung menyerang. Jika belum, unit bergerak mendekati target sambil menghindari penumpukan dengan unit kawan. Ketika tidak ditemukan target musuh, *Splasher* bergerak secara eksploratif menuju petak kosong atau area musuh terdekat.

4. Sistem Greedy Pergerakan

   Pemilihan arah gerak pada unit menggunakan sistem *greedy scoring*. Setiap arah yang mungkin dievaluasi berdasarkan beberapa faktor:

   - jarak terhadap target,
   - penalti terhadap petak bercat musuh,
   - bonus untuk mempertahankan arah sebelumnya,
   - penalti untuk menghindari gerakan bolak-balik,
   - penalti jika posisi baru terlalu dekat dengan unit kawan.
   - variasi acak kecil untuk mengurangi pola gerak deterministik.

   Arah dengan skor tertinggi dipilih sebagai langkah berikutnya. Pendekatan ini membuat unit cenderung bergerak stabil menuju target tanpa perhitungan jalur global.

5. Karakteristik Heuristic

   Heuristik *Ruinova* termasuk strategi greedy karena setiap keputusan selalu diambil berdasarkan keuntungan lokal terbaik pada giliran saat itu, tanpa menyimpan perencanaan jangka panjang. Prioritas utama selalu diberikan pada aksi yang memberikan dampak langsung terbesar, yaitu pembangunan *tower*, perluasan wilayah, dan pembersihan area lawan.

6. Analisis Efisiensi Program

   Dari sisi efisiensi, heuristic *Ruinova* memiliki kompleksitas keputusan yang rendah karena seluruh aksi ditentukan berdasarkan informasi lokal pada radius pengamatan unit tanpa melakukan pencarian jalur global. Evaluasi kandidat gerak hanya dilakukan pada delapan arah di sekitar unit, sehingga biaya komputasinya tetap rendah meskipun jumlah unit meningkat. Pendekatan ini sesuai dengan konsep permainan **turn-based** yang menuntut pengambilan keputusan cepat pada setiap giliran.

7. Analisis Efektivitas Program

   Dari sisi efektivitas, strategi ini mampu menggabungkan dua tujuan utama secara bersamaan, yaitu ekspansi wilayah dan pembangunan *tower*. Prioritas terhadap *ruin* memungkinkan pembentukan infrastruktur baru lebih cepat dibanding pewarnaan acak, sehingga produksi unit dapat meningkat lebih awal. Pembagian peran antara *Soldier* dan *Splasher* juga membuat distribusi tugas lebih terarah: *Soldier* fokus pada pembangunan dan pewarnaan area netral, sedangkan *Splasher* menekan dominasi wilayah lawan melalui serangan area. Namun, karena seluruh keputusan bersifat lokal, strategi ini memiliki keterbatasan dalam menghadapi kondisi yang memerlukan koordinasi global atau prediksi jangka panjang.

3.2.2 Penerapan 2: Embege

Algoritma pada program ini menggunakan pendekatan heuristic *greedy by objective ruin* dengan koordinasi antara Tower dan Soldier. Ide utamanya adalah menjadikan pembangunan tower sebagai prioritas utama, sehingga seluruh keputusan lokal selalu diarahkan untuk menemukan ruin, mengirim Soldier ke ruin, lalu menyelesaikan pembangunan tower secepat mungkin.

1. Peran Tower

   Tower berfungsi sebagai pusat koordinasi. Setiap turn, tower membaca pesan yang masuk lalu menyimpan informasi ruin ke dalam memori lokal. Untuk setiap ruin, tower menyimpan lokasi, status, jumlah Soldier yang sedang ditugaskan, dan waktu pembaruan terakhir.

   Setelah itu, tower memilih ruin terbaik menggunakan fungsi greedy. Ruin yang sedang dibangun diberi prioritas lebih tinggi daripada ruin yang baru ditemukan. Selain itu, ruin dengan worker yang lebih sedikit dan jarak yang lebih dekat juga lebih diprioritaskan. Setelah ruin terbaik dipilih, tower mengirim assignment kepada Soldier terdekat yang dapat dihubungi. Dengan cara ini, tower mengatur distribusi Soldier agar tidak menumpuk berlebihan pada satu target

2. Peran Soldier

   Soldier berfungsi sebagai pencari dan pembangun tower. Pada awal turn, Soldier membaca assignment dari tower. Jika mendapat target resmi, Soldier akan fokus menuju target tersebut. Jika belum mendapat assignment, Soldier akan mencari ruin yang terlihat di sekitarnya. Ruin yang ditemukan sendiri disimpan sebagai target sementara, lalu dilaporkan ke tower.

   Setelah target ditentukan, Soldier bergerak menuju ruin. Ketika sudah dekat, Soldier mulai menandai pattern tower, mengecat tile yang belum sesuai, lalu

menyelesaikan pembangunan tower. Jika tower berhasil selesai dibangun, Soldier mengirim pesan selesai ke tower dan kembali siap menerima target berikutnya.

3. Pemilihan Ruin

   Greedy utama program terletak pada proses pemilihan ruin. Tower selalu memilih ruin yang paling menguntungkan secara lokal berdasarkan empat faktor:

   - status ruin
   - pembaruan terkahir
   - jumlah soldier yang sedangn membangun
   - jarak dari tower

   Dengan pendekatan ini, sistem lebih memilih menyelesaikan ruin yang sudah mulai dikerjakan daripada terus membuka target baru. Hal ini membuat pembangunan tower lebih stabil dan mengurangi risiko split focus.

4. Komunikasi

   Komunikasi digunakan untuk menjaga koordinasi antar unit. Soldier mengirim pesan saat:

   - menemukan ruin
   - mulai mengerjakan ruin
   - selesai membangun tower

   Sebaliknya, tower mengirim pesan assignment ke Soldier. Dengan mekanisme ini, tower menjadi pusat memori dan kontrol, sedangkan Soldier menjadi eksekutor di lapangan.

5. Analisis Efisiensi Program Ada beberapa hal yang membuat strategi ini cukup efisien:

   - Hanya memakai satu jenis unit, sehingga manajemen strategi lebih hemat dan tidak rumit,
   - Adanya worker limit pada ruin mencegah terlalu banyak Soldier terbuang di satu target,
   - Target yang sudah dikerjakan diprioritaskan, sehingga progress tidak sering terbuang sia-sia.

6. Analisis Efektivitas Program

   Secara efisiensi program ini dapat efisien untuk menemukan ruin serta mengkoordinasikan soldier untuk mengecat tile di sekitar ruin dan membangunnya. Program ini secara efisien dapat membangun tower dari ruin secara satu per-satu.

3.2.3 Penerapan 3: Strategi *Jack of All Trades*

Strategi ini menerapkan pendekatan *greedy by local role optimization*, di mana setiap jenis entitas mengambil keputusan terbaik secara lokal berdasarkan perannya masing-masing tanpa koordinasi global. Strategi ini memiliki paling banyak lapisan *greedy* karena seluruh entitas Tower, Soldier, Splasher, dan Mopper masing-masing menjalankan heuristik yang berbeda secara paralel setiap giliran.

1. Peran Tower

   Tower berperan sebagai pusat produksi unit. Setiap giliran, Tower memilih tipe unit yang akan di-*spawn* menggunakan *greedy by composition deficit*: tipe unit dengan selisih terbesar antara jumlah aktual dan rasio target yang dipilih. Pada fase `EARLY`, rasio targetnya adalah 75% Soldier dan 25% Splasher tanpa Mopper sama sekali. Pada fase `MID` dan `LATE`, rasionya bergeser menjadi sekitar 47% Soldier, 48% Splasher, dan 5% Mopper.

   Di luar urusan *spawn*, Tower juga secara greedy memilih unit sekutu dengan paint paling kritis untuk diisi ulang, dan menyerang unit musuh terlemah dalam jangkauan setiap giliran.

2. Peran Soldier

   Soldier berperan untuk eksplorasi, pewarnaan, dan pembangunan tower. Setiap giliran, Soldier menentukan *state*-nya secara greedy berdasarkan kondisi lokal: jika paint habis maka *refill*, jika ada Mopper musuh maka *flee*, jika ada ruin terdekat maka *build*, dan jika tidak ada keduanya maka *explore*.

   Pada state `BUILD`, Soldier memilih ruin target dengan memilih ruin yang belum ditempati tower terdekat dari posisinya saat ini. Tipe tower dipilih menggunakan *greedy by balance*: Soldier menghitung jumlah Paint Tower dan Money Tower sekutu yang terlihat, lalu memilih tipe yang jumlahnya lebih sedikit. Setelah target ditentukan, Soldier bergerak menggunakan `greedyMove` yang mengevaluasi semua 8 arah dan memilih arahdengan skor tertinggi berdasarkan jarak ke target, momentum arah, penalti balik ke posisi sebelumnya, dan penalti tile bercat musuh.

   Pada state `EXPLORE`, Soldier menggunakan `greedyPaint` yang memilih tile terbaik untuk dicat dalam radius 9 berdasarkan prioritas: tile bertanda sekutu yang belum sesuai (skor 30), tile bercat musuh (skor 15), dan tile kosong (skor 8). Pergerakan pada state ini menggunakan *sector cycling* untuk menjelajahi area yang belum terjangkau.

3. Peran Splasher

   Splasher berperan sebagai unit agresif yang difokuskan pada tekanan wilayah. Splasher langsung menyerang tile pertama yang valid dalam radius 4, baik tile bercat musuh maupun tile yang ditempati unit musuh. Pendekatan ini mengorbankan efisiensi paint per serangan demi kecepatan eksekusi dan penghematan *bytecode*.

   Untuk pergerakan, Splasher mengikuti *sector cycling* 4×4 sebagai eksplorasi dasar. Satu-satunya kondisi yang mengubah perilaku Splasher adalah kehadiran Mopper musuh, di mana Splasher akan melarikan diri ke arah berlawanan dari centroid posisi Mopper musuh.

4. Peran Mopper

   Mopper berperan sebagai unit pembersih dan distributor paint. Setiap giliran, Mopper menentukan *state*-nya menggunakan *greedy by priority chain*: adanya musuh memicu state `HUNT`, paint kritis memicu `RECHARGE`, paint melimpah di fase non-`EARLY` memicu `DISTRIBUTE`, dan jika tidak ada kondisi khusus maka `EXPLORE` yang juga memicu `SCRUB` jika terdeteksi cat musuh di sekitarnya.

   Pada state `HUNT`, Mopper memilih target musuh terdekat secara greedy lalu melakukan ayunan *mop* jika memungkinkan. Pada state `SCRUB`, Mopper mengevaluasi semua 4 arah kardinal dan memilih arah ayunan dengan nilai terbanyak berdasarkan jumlah unit musuh dan tile bercat musuh yang dapat terkena dalam satu ayunan. Pada state `DISTRIBUTE`, Mopper mencari unit sekutu dengan paint di bawah 50% kapasitas dan bergerak menuju yang terdekat untuk ditransfer.

5. Analisis Efisiensi Program

   Secara efisiensi, program ini dapat dengan efisien mendapatkan daerah terutama di early game. Keuntungan terbesar dari program ini adalah dioptimalisasinya setiap jenis bot dan tower dengan *sensing* dilakukan satu kali di awal giliran dan keputusan diambil langsung tanpa iterasi berulang. Pengecekan tower terdekat juga dibatasi setiap 10 giliran untuk mengurangi beban komputasi.

6. Analisis Efektivitas Program

   Efektivitas program ini datang dari kesederhanaan koordinasi dan keseluruhan entitas terdapat optimasi. Terdapat kelemahan yaitu kurang dalamnya salah satu entitas karena diversifikasi tersebut, pada *case* ini adalah bagian splasher.

3.3 Pemilihan Strategi Greedy Utama

Strategi greedy utama yang dipilih untuk diimplementasikan dalam program adalah *Ruinova*. Strategi ini dipilih karena mampu mempercepat pencapaian dominasi wilayah dengan menggabungkan dua proses penting secara paralel, yaitu ekspansi melalui pembangunan *tower* pada *ruin* dan pewarnaan area sekitar secara terus-menerus. Dibandingkan strategi yang terlalu terfokus pada koordinasi pembangunan saja atau strategi yang membagi perhatian ke terlalu banyak peran, *Ruinova* memberikan laju pertumbuhan area yang lebih stabil sehingga lebih berpotensi mencapai target penguasaan lebih dari 70% peta dalam waktu yang lebih cepat.

Selain itu, *Ruinova* dinilai memiliki potensi kemenangan yang lebih tinggi karena pembagian peran unit berlangsung sederhana namun efektif. *Soldier* memperluas wilayah sambil membangun *tower* baru, sedangkan *Splasher* menekan wilayah lawan dengan membersihkan cat musuh pada area bernilai tinggi. Kombinasi ini memungkinkan tekanan ofensif dan pertumbuhan sumber daya berlangsung bersamaan, sehingga strategi tetap kompetitif baik pada fase awal maupun pertengahan permainan dengan biaya komputasi yang rendah.

4 Implementasi dan Pengujian

4.1 Implementasi Algoritma Greedy pada Bot

4.1.1 Ruinova

```
function RunTower(rc : RobotController) -> void
{ Mengendalikan tower untuk upgrade, menyerang musuh, dan menentukan produksi unit }

Deklarasi:
    enemies : array of RobotInfo
    build : UnitType
    spawn : MapLocation
    d : Direction

Algoritma:
    { 1. Upgrade tower setelah mid game }
    if round > 300 and tower dapat di-upgrade then
        upgrade tower

    { 2. Menyerang musuh terdekat jika ada }
    enemies <- sense nearby enemy robots
    if enemies tersedia dan dapat diserang then
        attack lokasi musuh pertama

    { 3. Menentukan unit berdasarkan fase permainan }
    if round < 400 then
        setiap 3 Soldier hasilkan 1 Splasher
    else if round <= 800 then
        setiap 2 Splasher hasilkan 1 Soldier
    else if jumlah tower < 25 then
        prioritaskan Splasher dengan Soldier periodik
    else
        hasilkan Splasher terus menerus

    { 4. Spawn unit pada tile kosong pertama }
    for each direction d do
        spawn <- lokasi tower + d
        if spawn valid then
            build robot
            keluar

function RunSoldier(rc : RobotController) -> void
{ Soldier mencari ruin, membangun tower, dan mengecat area kosong }

Deklarasi:
    ruin : MapLocation
    tower : UnitType
    tile : MapInfo
```

```
Algoritma:
    { 1. Refill paint jika rendah }
    if paint < 100 then
        cari tower ally terdekat
        transfer paint

    { 2. Cari ruin terdekat yang belum memiliki tower }
    ruin <- null
    for each visible tile do
        if tile memiliki ruin dan belum ada tower then
            ruin <- lokasi ruin
            keluar

    { 3. Jika ruin ditemukan, bergerak menuju ruin }
    if ruin != null then
        if jarak ke ruin > 4 then
            greedyMove ke ruin
            keluar

    { 4. Tentukan tipe tower }
        if jumlah tower genap then
            pilih Money Tower
        else
            pilih Paint Tower

    { 5. Tandai pola tower }
        jika pattern dapat dibuat maka tandai pattern

    { 6. Warnai tile sesuai pattern }
        untuk setiap tile sekitar ruin:
            jika mark tidak sesuai paint maka
                attack tile tersebut

    { 7. Selesaikan tower jika pattern lengkap }
        jika tower dapat diselesaikan maka
            complete tower pattern
            keluar

    { 8. Jika tidak ada ruin, cat tile kosong di posisi sendiri }
    if tile sekarang kosong then
        attack posisi sendiri

    { 9. Eksplorasi area baru }
    greedyExplore

function RunSplasher(rc : RobotController) -> void
{ Splasher mencari cluster paint musuh lalu menyerang area bernilai tertinggi }

Deklarasi:
    bestTarget : MapLocation
```

```
    bestScore : integer
    score : integer

Algoritma:
    { 1. Evaluasi semua tile enemy paint }
    bestTarget <- null
    bestScore <- nilai minimum

    for each visible tile do
        if tile adalah enemy paint then
            hitung jumlah enemy paint sekitar
            score <- cluster * 15 - distance * 3

            jika score lebih besar maka
                simpan target terbaik

    { 2. Serang target terbaik }
    if bestTarget tersedia then
        attack bestTarget
        greedyMoveSplasher ke target
        keluar

    { 3. Jika tidak ada target, cat posisi sendiri }
    if tile sekarang belum ally then
        attack posisi sendiri

    { 4. Lanjut eksplorasi }
    greedyMoveSplasher tanpa target

function GreedyMove(rc : RobotController, target : MapLocation, avoidEnemy : boolean)
{ Memilih arah gerak terbaik berdasarkan skor lokal }

Deklarasi:
    bestDir : Direction
    bestScore : integer
    score : integer

Algoritma:
    bestScore <- nilai minimum

    for each direction do
        jika arah dapat dilalui then
            hitung jarak baru ke target
            score <- -jarak baru * 5

            jika mendekat tambah skor
            jika arah sama seperti sebelumnya tambah skor
            jika kembali ke posisi lama kurangi skor

            jika tile enemy dan avoidEnemy aktif
```

```
                penalti besar

            jika ada ally di tile tujuan
                penalti

            jika score lebih besar maka
                simpan arah terbaik

    jika bestDir tersedia then
        move ke bestDir

    function GreedyExplore(rc : RobotController, avoidEnemy : boolean) -> void
{ Eksplorasi lokal untuk mencari area kosong atau aman }

Deklarasi:
    bestDir : Direction
    score : integer

Algoritma:
    untuk setiap direction:
        jika bisa bergerak:
            prioritaskan arah sebelumnya
            hindari tile enemy
            prioritaskan tile kosong
            hindari ally padat

    pilih arah skor tertinggi lalu bergerak
```

4.1.2 Embege

```
function RunTower(rc : RobotController) -> void
{ Mengendalikan logika menara untuk membaca pesan, merekam ruin, membangun prajurit,

Deklarasi:
    messages : array of Message
    m : Message
    msg, type, idx : integer
    loc, spawn : MapLocation
    dir : Direction

Algoritma:
    { 1. Membaca pesan masuk dan memperbarui database reruntuhan }
    messages <- rc.readMessages(-1)
    for each m in messages do
        msg <- m.getBytes()
        type <- decodeType(msg)
        loc <- decodeLoc(msg)

        if isValidEncodedLoc(loc) then
            if type = MSG_DISCOVER then
```

```
                addOrUpdateRuin(loc, RUIN_DISCOVERED, rc.getRoundNum())
                maybeBroadcast(rc, msg)
            else if type = MSG_BUILDING then
                addOrUpdateRuin(loc, RUIN_BUILDING, rc.getRoundNum())
            else if type = MSG_DONE then
                idx <- findRuinIndex(loc)
                addOrUpdateRuin(loc, RUIN_DONE, rc.getRoundNum())
                if idx != -1 then
                    ruinWorkers[idx] <- 0
                maybeBroadcast(rc, msg)

    cleanupStaleRuinData(rc)

    { 2. Melakukan Upgrade jika memungkinkan }
    if rc.canUpgradeTower(rc.getLocation()) then
        rc.upgradeTower(rc.getLocation())

    { 3. Membangun Soldier }
    for each dir in shuffledDirections() do
        spawn <- rc.getLocation().add(dir)
        if rc.canBuildRobot(SOLDIER, spawn) then
            rc.buildRobot(SOLDIER, spawn)
            break

    { 4. Menugaskan reruntuhan terbaik ke Soldier terdekat }
    assignRuinToNearbySoldiers(rc)

    { 5. Menyerang musuh jika ada }
    tryTowerAttack(rc)

function RunSoldier(rc : RobotController) -> void
{ Mengendalikan logika prajurit untuk mencari, menuju, dan membangun reruntuhan }

Deklarasi:
    visibleRuin, myTarget : MapLocation

Algoritma:
    readSoldierMessages(rc)
    homeTower <- findNearestReachableTowerForMessaging(rc)
    visibleRuin <- findVisibleUntoweredRuin(rc)

    if visibleRuin != null then
        if assignedRuin = null then
            tentativeRuin <- visibleRuin

        if rc.getRoundNum() - lastReportedRound >= 5 then
            tryReportToTower(rc, MSG_DISCOVER, visibleRuin)

    { Menentukan target reruntuhan }
    if assignedRuin != null then
```

```
        myTarget <- assignedRuin
    else if tentativeRuin != null then
        myTarget <- tentativeRuin
    else
        myTarget <- null

    { Jika tidak ada target, lakukan eksplorasi }
    if myTarget = null then
        randomExplore(rc)
        return

    { Jika target sudah tidak valid (misal sudah dibangun musuh) }
    if not isPotentiallyValidRuinTarget(rc, myTarget) then
        assignedRuin <- null
        tentativeRuin <- null
        randomExplore(rc)
        return

    { Jika masih jauh dari reruntuhan, bergerak mendekat }
    if rc.getLocation().distanceSquaredTo(myTarget) > 8 then
        PathFinding(rc, myTarget)
        return

    { Jika sudah dekat, laporkan status pembangunan }
    if rc.getRoundNum() - lastBuildingReportRound >= 5 then
        tryReportToTower(rc, MSG_BUILDING, myTarget)
        lastBuildingReportRound <- rc.getRoundNum()

    { Lakukan pengecatan/pembangunan }
    if workOnRuin(rc, myTarget) = true then
        return

    { Reposisi jika terhalang saat membangun }
    PathFinding(rc, myTarget)

function PathFinding(dest : MapLocation) -> void
    { Menavigasi robot menuju titik tujuan menggunakan algoritma Bug2 }

    Deklarasi:
        dir : Direction
        i : integer

    Algoritma:
        if rc.getLocation() = dest then
            return
        endif

        if target = null or target != dest then
            target <- dest
            prevDest <- target
```

```
            line <- createLine(target, rc.getLocation())
            isTracing <- false
        endif

        if isTracing = false then
            dir <- rc.getLocation().directionTo(target)
            if dir != CENTER and rc.canMove(dir) then
                rc.move(dir)
            else
                isTracing <- true
                obstacleStarDist <- rc.getLocation().distanceSquaredTo(target)
                tracingDirection <- dir
            endif
        else
            if line.contains(rc.getLocation()) and rc.getLocation().distanceSquaredTo
                isTracing <- false
            else
                if tracingDirection != CENTER and rc.canMove(tracingDirection) then
                    rc.move(tracingDirection)
                    tracingDirection <- tracingDirection.rotateRight().rotateRight()
                else
                    for i <- 0 to 7 do
                        tracingDirection <- tracingDirection.rotateLeft()
                        if tracingDirection != CENTER and rc.canMove(tracingDirection
                            rc.move(tracingDirection)
                            tracingDirection <- tracingDirection.rotateRight().rotate
                            break
                        endif
                    endfor
                endif
            endif
        endif
```

4.1.3 Jack of All Trades

```
function MainLoop(rc : RobotController) -> void
{ Mengeksekusi loop logika utama untuk robot, mengatur fase permainan dan
mendelegasikan tugas ke spesifik unit }

Deklarasi:
    nearbyAllies : RobotInfo[]
    ally : RobotInfo

Algoritma:
    turnCount <- turnCount + 1

    { Menentukan Fase Permainan }
    if turnCount <= 300 then
        currentPhase <- EARLY
```

```
    else if turnCount <= 1000 then
        currentPhase <- MID
    else
        currentPhase <- LATE
    endif

    { Throttling: Update tower sekutu terdekat hanya setiap 10 turn untuk
    hemat komputasi }
    if rc.getType() != TOWER and (turnCount mod 10) = 1 then
        closestRefillTower <- null
        nearbyAllies <- rc.senseNearbyRobots(-1, rc.getTeam())

        for ally in nearbyAllies do
            if ally.getType() = TOWER then
                if closestRefillTower = null or
                rc.getLocation().distanceSquaredTo(ally.getLocation()) <
                rc.getLocation().distanceSquaredTo(closestRefillTower) then
                    closestRefillTower <- ally.getLocation()
                endif
            endif
        endfor
    endif

    { Mendelegasikan tugas berdasarkan tipe unit }
    switch rc.getType() do
        case SOLDIER:
            SoldierLogic(rc)
        case SPLASHER:
            SplasherLogic(rc)
        case MOPPER:
            MopperLogic(rc)
        default:
            TowerLogic(rc)
    endswitch

function ExploreByPhase(rc : RobotController) -> void
{ Menjelajahi peta menggunakan algoritma penentuan target berbasis
pembagian sektor }

Deklarasi:
    shouldRetarget : boolean

Algoritma:
    shouldRetarget <- false

    { Cek apakah perlu mencari target sektor baru }
    if exploreTarget = null or rc.getLocation().distanceSquaredTo(exploreTarget)
    <= 9 or rc.getRoundNum() - exploreRetargetRound >= 18 then
        shouldRetarget <- true
    endif
```

```
    if shouldRetarget = true then
        exploreVisitCount <- exploreVisitCount + 1
        exploreTarget <- getSectorTarget(rc, exploreVisitCount)
        exploreRetargetRound <- rc.getRoundNum()
    endif

    PathFinding(exploreTarget)

function MopperLogic(rc : RobotController) -> void
{ Mengelola perilaku unit Mopper termasuk memburu musuh, membersihkan cat,
mengisi ulang, dan distribusi cat }

Deklarasi:
    enemies : RobotInfo[]
    paintThreshold : integer
    bestEnemy : RobotInfo
    swingDir, bestSwingDir, dir : Direction
    closestEnemyPaint : MapLocation
    tile : MapInfo
    maxHit, hits : integer

Algoritma:
    enemies <- rc.senseNearbyRobots(-1, rc.getTeam().opponent())

    { Penentuan State berdasarkan prioritas }
    if currentPhase = EARLY then
        paintThreshold <- 15
    else
        paintThreshold <- 25
    endif

    if enemies.length > 0 then
        currentState <- HUNT
    else if rc.getPaint() < paintThreshold then
        currentState <- RECHARGE
    else if currentPhase != EARLY and rc.getPaint() > 140 then
        currentState <- DISTRIBUTE
    else
        currentState <- EXPLORE
        for tile in rc.senseNearbyMapInfos(9) do
            if tile.getPaint().isEnemy() then
                currentState <- SCRUB
                break
            endif
        endfor
    endif

    { Eksekusi State yang telah ditentukan }
    switch currentState do
```

```
case HUNT:
    bestEnemy <- getClosestEnemy(enemies)
    swingDir <- rc.getLocation().directionTo(bestEnemy.getLocation())
    if rc.isActionReady() and rc.canMopSwing(swingDir) then
        rc.mopSwing(swingDir)
    else
        PathFinding(bestEnemy.getLocation())
    endif

case RECHARGE:
    if closestRefillTower != null and
    rc.getLocation().distanceSquaredTo(closestRefillTower) > 2 then
        PathFinding(closestRefillTower)
    else if closestRefillTower = null then
        ExploreByPhase(rc)
    endif

case SCRUB:
    closestEnemyPaint <- getClosestEnemyPaintTile()
    if closestEnemyPaint != null then
        if rc.isActionReady() then
            bestSwingDir <- null
            maxHit <- 0
            { Cari arah ayunan pel dengan hit musuh terbanyak }
            for dir in {NORTH, SOUTH, EAST, WEST} do
                if rc.canMopSwing(dir) then
                    hits <- calculateSwingHits(dir)
                    if hits > maxHit then
                        maxHit <- hits
                        bestSwingDir <- dir
                    endif
                endif
            endfor

            if bestSwingDir != null and maxHit > 0 then
                rc.mopSwing(bestSwingDir)
                return
            endif

            attackClosestEnemyPaintTile()
        endif
        PathFinding(closestEnemyPaint)
    endif

case DISTRIBUTE:
    for ally in rc.senseNearbyRobots(-1, rc.getTeam()) do
        if ally.getType() != TOWER and ally.getPaintAmount() <
        ally.getType().paintCapacity * 0.5 then
            if rc.getLocation().distanceSquaredTo(ally.getLocation()) <= 2
```

```
                    and rc.canTransferPaint(ally.getLocation(), 20) then
                        rc.transferPaint(ally.getLocation(), 20)
                    else
                        PathFinding(ally.getLocation())
                    endif
                    return
                endif
            endfor
            ExploreByPhase(rc)

        case EXPLORE:
            if rc.isActionReady() then
                attackNearbyEnemyPaint()
            endif
            ExploreByPhase(rc)
    endswitch

function SoldierLogic(rc : RobotController) -> void
{ Mengelola perilaku unit Soldier termasuk membangun tower, kabur dari Mopper,
dan menjelajah }

Deklarasi:
    enemies : RobotInfo[]
    ruins : MapLocation[]
    needsToFlee, isPrimary : boolean
    enemy : RobotInfo
    avgX, avgY, mopperCount : integer
    awayDir : Direction
    ruinLoc : MapLocation

Algoritma:
    enemies <- rc.senseNearbyRobots(-1, rc.getTeam().opponent())
    ruins <- rc.senseNearbyRuins(-1)

    { Reset target ruin jika tower sekutu sudah berdiri di sana }
    if committedRuin != null and rc.canSenseRobotAtLocation(committedRuin) then
        if rc.senseRobotAtLocation(committedRuin).getType() = TOWER then
            committedRuin <- null
            committedType <- null
        endif
    endif

    { Penentuan State }
    if rc.getPaint() <= 50 then
        state <- REFILL
    else
        needsToFlee <- false
        for enemy in enemies do
            if enemy.getType() = MOPPER then
                needsToFlee <- true
```

```
        endif
    endfor

    if needsToFlee = true then
        state <- FLEE
    else if getBuildTarget(rc, ruins) != null then
        state <- BUILD
    else
        state <- EXPLORE
    endif
endif

{ Eksekusi State }
switch state do
    case REFILL:
        if closestRefillTower != null and
        rc.getLocation().distanceSquaredTo(closestRefillTower) > 2 then
            PathFinding(closestRefillTower)
        else if closestRefillTower = null then
            ExploreByPhase(rc)
        endif

    case FLEE:
        avgX <- 0
        avgY <- 0
        mopperCount <- 0
        for enemy in enemies do
            if enemy.getType() = MOPPER then
                avgX <- avgX + enemy.getLocation().x
                avgY <- avgY + enemy.getLocation().y
                mopperCount <- mopperCount + 1
            endif
        endfor

        if mopperCount > 0 then
            { Bergerak menjauhi titik tengah (centroid) dari Mopper musuh }
            awayDir <- rc.getLocation().
            (new MapLocation(avgX / mopperCount,
            avgY / mopperCount)).opposite()
            PathFinding(rc.getLocation().add(awayDir)
            .add(awayDir).add(awayDir))
        endif

    case BUILD:
        ruinLoc <- getBuildTarget(rc, ruins)
        if ruinLoc = null then
            greedyPaint(rc)
            ExploreByPhase(rc)
            return
```

```
            endif

            { Cek apakah Soldier ini adalah pembangun utama berdasarkan
            jarak dan ID }
            isPrimary <- checkPrimaryBuilderStatus(rc, ruinLoc)

            if isPrimary = false and currentPhase != LATE then
                { Menyebar agar tidak menumpuk di satu titik }
                applyAntiClumpingDispersal(rc, ruinLoc)
                greedyPaint(rc)
                return
            endif

            if committedType = null then
                committedType <- determineBalancingTowerType()
            endif

            if canCompleteTowerPattern(committedType, ruinLoc) then
                completeTowerPattern(committedType, ruinLoc)
                return
            endif

            if isPatternMarked(ruinLoc) = true then
                if rc.getLocation().distanceSquaredTo(ruinLoc) > 8 then
                    PathFinding(ruinLoc)
                    paintNetworkTowards(rc, ruinLoc, 30)
                else
                    paintMissingPatternTiles(rc, ruinLoc)
                endif
            else
                markTowerPattern(committedType, ruinLoc)
            endif

        case EXPLORE:
            if rc.isActionReady() then
                greedyPaint(rc)
            endif
            if rc.isMovementReady() then
                ExploreByPhase(rc)
            endif
    endswitch

function SplasherLogic(rc : RobotController) -> void
{ Mengelola perilaku serangan ofensif dan manuver menghindar dari unit Splasher }

Deklarasi:
    enemies : RobotInfo[]
    enemy : RobotInfo
    needsToFlee : boolean
    sumX, sumY, mopperCount : integer
```

```
    awayDir : Direction
    attackTarget : MapLocation
    tile : MapInfo

Algoritma:
    enemies <- rc.senseNearbyRobots(-1, rc.getTeam().opponent())
    needsToFlee <- false

    { Cek ancaman Mopper }
    for enemy in enemies do
        if enemy.getType() = MOPPER then
            needsToFlee <- true
        endif
    endfor

    if needsToFlee = true then
        { Lari dari Mopper }
        sumX <- 0
        sumY <- 0
        mopperCount <- 0

        for enemy in enemies do
            if enemy.getType() = MOPPER then
                sumX <- sumX + enemy.getLocation().x
                sumY <- sumY + enemy.getLocation().y
                mopperCount <- mopperCount + 1
            endif
        endfor

        if mopperCount > 0 then
            awayDir <- rc.getLocation().directionTo
            (new MapLocation(sumX / mopperCount, sumY / mopperCount)).opposite()
            PathFinding(rc.getLocation().add(awayDir).add(awayDir).add(awayDir))
        endif
    else
        { Fokus Serangan Murni }
        if rc.isActionReady() then
            attackTarget <- null
            for tile in rc.senseNearbyMapInfos(4) do
                if rc.canAttack(tile.getMapLocation()) then
                    if tile.getPaint().isEnemy() or
                    rc.senseRobotAtLocation(tile.getMapLocation()) != null then
                        attackTarget <- tile.getMapLocation()
                        break
                    endif
                endif
            endfor

            if attackTarget != null then
```

```
                rc.attack(attackTarget)
            endif
        endif

        ExploreByPhase(rc)
    endif

function TowerLogic(rc : RobotController) -> void
{ Mengelola perilaku Tower termasuk menyerang musuh, distribusi cat ke
sekutu, dan memproduksi unit baru }

Deklarasi:
    enemies, allies : RobotInfo[]
    weakestEnemy, enemy, ally : RobotInfo
    minHealth, criticalThreshold, transferAmount : integer
    totalUnits, bestDeficitScore : integer
    buildType : UnitType
    spawnDir : Direction

Algoritma:
    enemies <- rc.senseNearbyRobots(-1, rc.getTeam().opponent())
    allies <- rc.senseNearbyRobots(-1, rc.getTeam())

    { Serangan Dasar }
    if rc.isActionReady() and enemies.length > 0 then
        weakestEnemy <- null
        minHealth <- 9999
        for enemy in enemies do
            if rc.canAttack(enemy.getLocation()) and enemy.getHealth()
            < minHealth then
                minHealth <- enemy.getHealth()
                weakestEnemy <- enemy
            endif
        endfor
        if weakestEnemy != null then
            rc.attack(weakestEnemy.getLocation())
        endif
    endif

    { Distribusi Cat }
    if currentPhase = EARLY then
        criticalThreshold <- 45
    else if currentPhase = MID then
        criticalThreshold <- 25
    else
        criticalThreshold <- 20
    endif

    for ally in allies do
        if ally.getPaintAmount() < 30 then
```

```
        transferAmount <- 30
    else
        transferAmount <- 20
    endif

    if ally.getType() != TOWER and ally.getPaintAmount() < criticalThreshold
    and rc.getPaint() > 90 and rc.canTransferPaint(ally.getLocation(), transferAm
        rc.transferPaint(ally.getLocation(), transferAmount)
        break
    endif
endfor

{ Produksi Unit (Spawning) }
if rc.isActionReady() = false or rc.getPaint() <= 150 then
    return
endif

buildType <- null
totalUnits <- max(1, soldierSpawnCount + splasherSpawnCount
+ mopperSpawnCount)

{ Pemilihan tipe unit berdasarkan rasio kebutuhan fase permainan }
if currentPhase = EARLY then
    if (75 * totalUnits - soldierSpawnCount * 100) >=
    (25 * totalUnits - splasherSpawnCount * 100) then
        buildType <- SOLDIER
    else
        buildType <- SPLASHER
    endif
else
    bestDeficitScore <- -9999
    if rc.getPaint() >= 200 and (47 * totalUnits - soldierSpawnCount * 100) >
    bestDeficitScore then
        bestDeficitScore <- 47 * totalUnits - soldierSpawnCount * 100
        buildType <- SOLDIER
    endif
    if rc.getPaint() >= 400 and (48 * totalUnits - splasherSpawnCount * 100) >
    bestDeficitScore then
        bestDeficitScore <- 48 * totalUnits - splasherSpawnCount * 100
        buildType <- SPLASHER
    endif
    if rc.getPaint() >= 200 and (5 * totalUnits - mopperSpawnCount * 100) >
    bestDeficitScore then
        bestDeficitScore <- 5 * totalUnits - mopperSpawnCount * 100
        buildType <- MOPPER
    endif
endif

if buildType != null then
```

```
    for spawnDir in ALL_DIRS do
        if rc.canBuildRobot(buildType, rc.getLocation().add(spawnDir)) then
            rc.buildRobot(buildType, rc.getLocation().add(spawnDir))

            { Update jumlah unit yang berhasil dibuat }
            if buildType = SOLDIER then
                soldierSpawnCount <- soldierSpawnCount + 1
            else if buildType = SPLASHER then
                splasherSpawnCount <- splasherSpawnCount + 1
            else
                mopperSpawnCount <- mopperSpawnCount + 1
            endif

            return
        endif
    endfor
endif
```

4.2 Struktur Data, Fungsi, dan Prosedur pada Strategi *Ruinova*

Strategi *Ruinova* menggunakan struktur program yang relatif sederhana dengan fokus utama pada keputusan lokal berbasis greedy untuk mempercepat ekspansi wilayah dan pembangunan tower. Seluruh keputusan dilakukan secara langsung berdasarkan informasi lingkungan sekitar tanpa penyimpanan memori global yang kompleks.

4.2.1 Struktur Data yang Digunakan

Beberapa struktur data utama yang digunakan dalam program adalah sebagai berikut:

- `int turnCount`

  Variabel ini menyimpan jumlah giliran permainan yang telah berjalan. Nilai ini bertambah setiap turn dan digunakan untuk membantu sinkronisasi perilaku global bot.

- `int spawnCounter`

  Variabel ini digunakan untuk menghitung jumlah unit yang telah dihasilkan oleh tower. Nilai ini menjadi dasar greedy spawning untuk menentukan apakah tower akan membangun *Soldier* atau *Splasher* sesuai fase permainan.

- `Random rng`

  Generator bilangan acak yang digunakan untuk memberi variasi kecil pada skor greedy movement agar unit tidak bergerak terlalu deterministik.

- `Direction myDir`

  Menyimpan arah gerak sebelumnya. Struktur ini digunakan untuk memberikan bonus skor pada arah yang sama sehingga gerakan unit lebih stabil dan tidak mudah berosilasi.

- `MapLocation lastLoc`

  Menyimpan posisi terakhir unit. Digunakan untuk mengurangi kemungkinan unit kembali ke posisi sebelumnya.

- `MapInfo[]`

  Struktur array ini diperoleh dari fungsi sensing dan digunakan untuk membaca kondisi tile sekitar, termasuk ruin, paint, dan mark pattern.

- `RobotInfo[]`

  Struktur array ini digunakan untuk membaca robot di sekitar unit, baik ally maupun enemy, untuk kebutuhan attack, refill paint, dan menghindari penumpukan posisi.

4.2.2 Fungsi Utama Program

Program terdiri atas beberapa fungsi utama yang masing-masing menangani perilaku entitas berbeda.

- `run(RobotController rc)`

  Fungsi utama program yang dijalankan setiap turn. Fungsi ini bertugas mengidentifikasi tipe unit lalu memanggil prosedur sesuai jenis robot:

  - `runSoldier()` untuk Soldier
  - `runSplasher()` untuk Splasher
  - `runTower()` untuk Tower

- `runTower(RobotController rc)`

  Fungsi ini mengatur seluruh keputusan tower:

  - melakukan upgrade setelah ronde tertentu,
  - menyerang musuh jika ada dalam jangkauan,
  - menentukan jenis unit yang dibangun berdasarkan fase permainan.

  Pemilihan unit dilakukan secara greedy berdasarkan *spawn ratio heuristic* agar early game fokus pada ekspansi ruin, sedangkan mid hingga late game fokus pada tekanan area.

- `runSoldier(RobotController rc)`

  Fungsi ini merupakan inti strategi *Ruinova*. Soldier:

  - mengambil paint dari tower jika paint rendah,
  - mencari ruin terdekat,
  - bergerak menuju ruin,
  - menandai pattern tower,
  - mengecat tile sesuai pattern,
  - menyelesaikan pembangunan tower.

Seluruh keputusan dilakukan secara greedy berdasarkan ruin terdekat yang tersedia.

- `runSplasher(RobotController rc)`

  Fungsi ini bertugas memberi tekanan wilayah dengan memilih area enemy paint terbaik berdasarkan skor lokal. Splasher mencari cluster enemy paint lalu memilih target dengan skor tertinggi.

4.2.3 Prosedur Greedy Movement

Movement dalam *Ruinova* menggunakan evaluasi lokal terhadap seluruh arah yang tersedia.

- `greedyMove(RobotController rc, MapLocation target, boolean avoidEnemy)`

  Digunakan oleh Soldier untuk bergerak menuju target ruin. Skor arah dihitung berdasarkan:

  - jarak baru ke target,
  - apakah arah mendekati target,
  - konsistensi arah sebelumnya,
  - penalti jika kembali ke posisi lama,
  - penalti jika tile musuh.

- `greedyExplore(RobotController rc, boolean avoidEnemy)`

  Digunakan ketika Soldier tidak menemukan ruin. Fungsi ini memprioritaskan tile kosong agar eksplorasi area berlangsung cepat.

- `greedyMoveSplasher(RobotController rc, MapLocation target)`

  Digunakan oleh Splasher untuk bergerak menuju area paint musuh atau menjelajah area kosong. Fungsi ini juga memberi penalti jika terlalu dekat dengan ally agar distribusi unit tetap menyebar.

4.2.4 Hubungan Struktur Program dengan Strategi Greedy

Seluruh struktur program *Ruinova* dirancang untuk mendukung greedy expansion yang cepat.

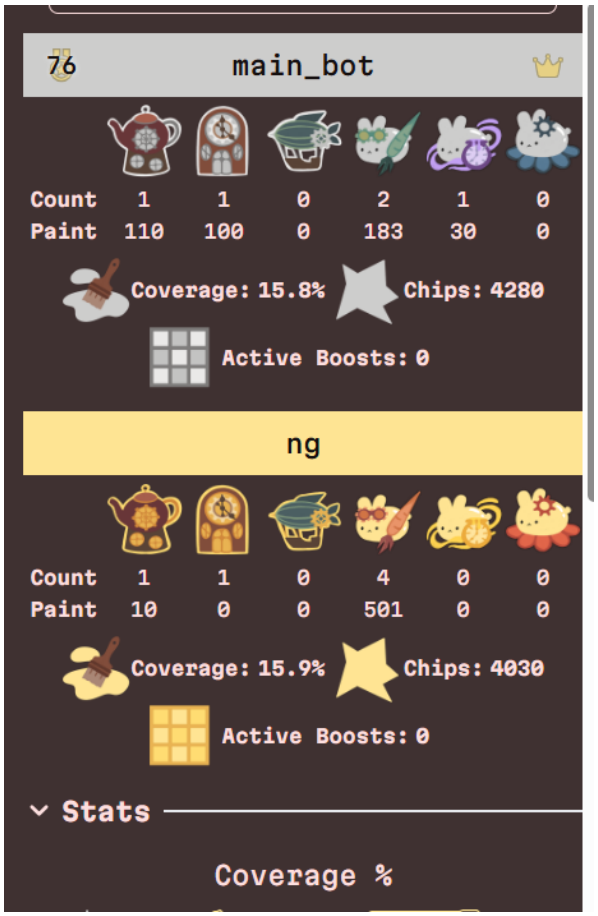
Pada early game, Soldier diprioritaskan agar ruin dapat ditemukan dan diselesaikan secepat mungkin. Hal ini memungkinkan area sekitar tower cepat ter-cover oleh paint ally dan memperbesar peluang menguasai lebih dari 70% area pada fase awal permainan.

Pada mid hingga late game, proporsi Splasher diperbesar agar tekanan terhadap enemy paint meningkat. Dengan cara ini, strategi tidak hanya membangun tower tetapi juga mempertahankan dominasi area. Pendekatan greedy lokal ini dipilih karena:

- biaya komputasi rendah,
- keputusan dapat diambil setiap turn tanpa memori kompleks,
- sangat efektif untuk ekspansi cepat pada map terbuka.

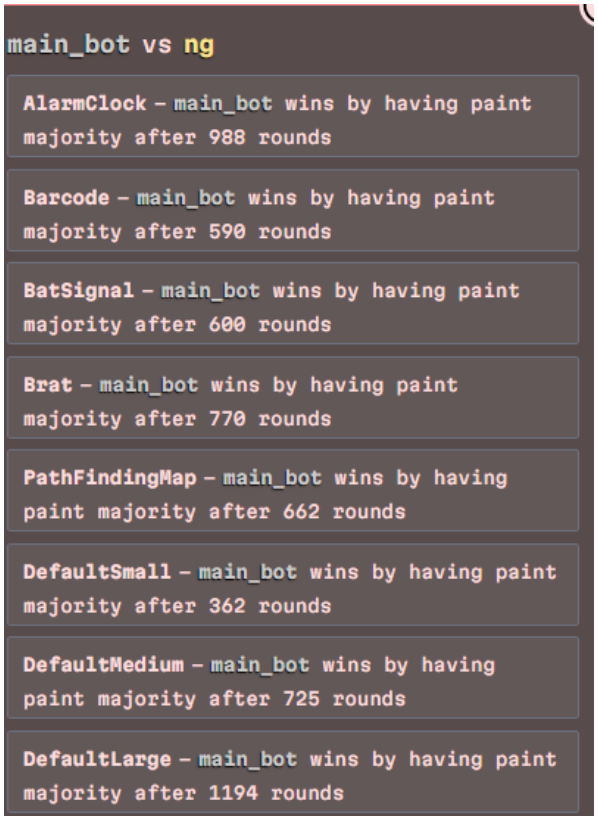
4.3 Pengujian

4.3.1 *Ruinova* vs *Jack of All Trades*



Gambar 3: *Ruinova* vs *Jack of All Trades*

Analisis : *Ruinova* atau *main_bot* hampir di semua map secara efektif menguasai tower-tower dari ruin, sedangkan *alt_2* (*Jack of All Trades*) hanya berfokus pada perluasan pengecatan. Dengan berhasilnya membangun beberapa tower di awal game, *Ruinova* berhasil meningkatkan *spawn* unit dengan masif. *Ruinova* lebih berfokus melakukan optimasi pada *spawn* splasher dan soldier ketimbang *Jack of All Trades* yang *spawn* semua type robot. Pengambilan keputusan secara paralel dari soldier dan splasher *Ruinova*, di mana soldier sangat berfokus untuk mencari ruin dan membuat tower, dan splasher yang sangat eksploratif dan menekan musuh membuat *Ruinova* berhasil melakukan ekspansi wilayah dengan cepat. Selain itu, splasher yang di-*spawn* oleh *Ruinova* sangat berfokus mengincar tile-tile lawan, membuat *Jack of All Trades* kehilangan banyak *coverage area* dalam waktu yang singkat, bisa dibuktikan karena di beberapa map game berjalan tidak sampai 2000 ronde karena *Ruinova* berhasil menguasai lebih dari 70% tile di map dengan sangat cepat.



Gambar 4: Bukti *Ruinova* menang kurang dari 2000 ronde

4.3.2 *Embege* vs *Jack of All Trades*

Gambar 5: *Embege* vs *Jack of All Trades*

Analisis : *Alt_1* atau *Embege* hanya berfokus pada pembangunan tower dan hanya memakai soldier selama game berlangsung, sehingga akan kalah dengan algoritma greedy bot yang memiliki type robot splasher yang menyerang tile dimiliki *Embege*. Namun *Embege* dalam beberapa map secara efektif membangun tower dari ruin satu persatu secara gotong royong dan sinkron antar soldier. *Embege* terlalu pasif dalam ekspansi wilayah, terlalu terpaku dengan ruin, sehingga ketika sudah tidak ada ruin lagi yang dapat dijangkau oleh soldier, *coverage area* dari *Embege* akan *stuck*. Sementara, *Jack of All Trades* lebih baik dalam perluasan wilayah. Melalui pengambilan keputusan secara lokal dari masing-masing unit, tiap-tiap unit dari *Jack of All Trades* bergerak dengan cepat sesuai tujuannya masing-masing, yaitu: soldier berfokus pada pembangunan ruin, splasher berfokus memperluas wilayah, dan mopper berfokus untuk mencuri *paint* musuh. Dengan pendekatan ini, *Jack of All Trades* berhasil memenangkan *battle* dengan telak.

4.3.3 *Ruinova* vs *Embege*

Gambar 6: *Ruinova* vs *Embege*

Analisis : Pada pertandingan 1 vs 1 antara *main_bot* alias *Ruinova* melawan *Embege*, *Ruinova* memenangkan pertandingan dengan skor telak, 76-0 tanpa balas. Di pertandingan ini, *Embege* yang sesuai namanya sangat "lapar" dalam membangun tower, hanya berfokus menikmati camilan-camilan ruin di sekitar tower *spawn* mereka. Karena terlalu asyik dengan reruntuhan-reruntuhan di sekitar mereka, unit-unit *Embege* tidak sadar tiba-tiba telah diserbu oleh pasukan splasher dari *Ruinova*. *Ruinova* yang lebih aktif dalam pembangunan tower, berhasil membangun tower dengan cepat, berhasil men-*spawn* splasher dalam jumlah yang banyak. Pasukan-pasukan splasher *Ruinova*, bagaikan anjing lapar melihat tile-tile *Embege* yang seolah-olah makan siangnya, dengan cepat merebut wilayah kekuasaan *Embege*, membuat game berakhir dengan cepat.

5 Penutup

5.1 Kesimpulan

Dalam proyek Tugas Besar 1 IF2211 Strategi Algoritma ini, kami berhasil mengimplementasikan algoritma greedy pada bot kami untuk permainan Battlecode

Kami berhasil membuat bot utama '**Ruinova**' dengan pendekatan heuristic bangun tower dengan ruin plus ekspansi wilayah musuh dengan splasher yang ganas. Dalam beberapa kondisi algoritma pada bot utama ini berhasil secara efektif dan efisien melakukan objektif utama game ini yaitu menguasai 70% tile pada suatu map.

Selain itu kami juga membuat 2 bot lain, yang satu menggunakan heuristic dengan membangun tower melalui koordinasi message antar tower dan soldier, sedangkan satu bot yang lain menggunakan heuritic greedy by 'local role optimization'

5.2 Saran dan Komentar

Pelaksanaan Tugas Besar 1 IF2211 Strategi Algoritma di Semester II (Genap) Tahun 2025/2026 merupakan pengalaman yang sangat bergizi bagi kami, terimakasih para asisten telah menyajikan tugas besar yang sangat epic ini, berikut komentar dari kelompok azzam is tired:

Azam

Asyik bang game dan tubesnya, semoga nggak demo pas lebaran plis T-T

Yusuf

Selamat hari raya idul fitri 1447 H, minal aidzin wal faidzin, mohon maaf lahir dan batin

Nathanael

Pemilihan game yang sangat keren dengan entitas yang banyak membuat program makin keren seperti Pak Bahlil yang menyelamatkan tanker pertamina.

5.3 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

[Yusuf Faishal Listyardi]

[Nathanael Gunawan]

[A. Fawwaz Azam Wicaksono]

6 Referensi

Kelas-kelas Battlecode 2025
PPT Pak Rinaldi Algoritma Greedy Bagian 1

7 Lampiran

https://github.com/ucupsss/Tubes1_azzam-is-tired.git